

Operating System Notebook — First Page

Welcome to **Gaurav's OS Prep Notebook** — a mission-driven space designed for serious aspirants of CUET-PG (MCA) and GATE CSE.

 **Notebook Launch Date:** 22 September 2025

 **Creator:** Gaurav (Allahabad University, B.Sc. Final Year)

This isn't just a bunch of OS notes. It's a launchpad.

What You'll Find Inside:

-  Hindi + English concept summaries
-  Visual diagrams and flowcharts
-  Quiz hooks after every topic
-  Weekly growth challenges
-  Verified digital twin with bonus content
-  Mentorship badges + Wall of Fame

Why This Notebook?

Because OS is not just theory — it's the backbone of your technical prep. And this notebook is built to make it:

- Clear
- Visual
- Practice-ready

How to Use:

- Start with the topic index
- Unlock each topic with summary + diagram + quiz
- Track your progress with monthly challenges
- Scan QR for bonus content and leaderboard access

OS Topic Index — Second Page

Welcome to your **OS Topic Index** — a curated roadmap designed to make your prep visual, bilingual, and quiz-ready.

◆ Unit 1: OS Basics

- What is Operating System?
- Types of Operating Systems
- Functions of OS
- System Calls

◆ Unit 2: Process Management

- Process vs Program
- PCB (Process Control Block)
- Context Switching
- Scheduling Algorithms (FCFS, SJF, RR, Priority)
- Multilevel Queue Scheduling

◆ Unit 3: Threads & Concurrency

- Threads vs Processes
- Multithreading
- Race Condition
- Critical Section Problem
- Semaphores & Mutex

◆ Unit 4: Memory Management

- Paging vs Segmentation
- Virtual Memory
- Page Replacement Algorithms (FIFO, LRU, Optimal)
- Thrashing

◆ Unit 5: File System & I/O

- File Types & Access Methods
- Directory Structures
- Disk Scheduling Algorithms (FCFS, SSTF, SCAN, C-SCAN)
- I/O Buffering

◆ Unit 6: Deadlock

- Deadlock Conditions
- Prevention, Avoidance, Detection
- Banker's Algorithm

"Har topic ke baad milega: Summary + Diagram + Quiz Hook + Shortcut Tag"

Let's unlock Unit 1 → **Start with: What is Operating System?**

📖 For the Reader — Third Page

Dear Reader,

Before you dive into the world of Operating Systems, take a moment to pause and reflect.

"An Operating System is not just software — it's how control, coordination, and clarity come together. Just like life." — Gaurav

This notebook is more than just technical prep. It's a mindset. Every page is designed to:

- Help you think like a system designer
- Build patience like a scheduler
- Handle pressure like a deadlock resolver

Whether you're preparing for CUET-PG, GATE, or just curious about how computers think — this notebook is your companion.

So flip the page, unlock the topic, and remember:

"Every process starts with a fork — choose yours wisely."

– Gaurav

🔒 Topic Unlock — What is Operating System?

Concept Summary:

An Operating System (OS) is a system software that acts as an interface between the user and the hardware. It manages resources like CPU, memory, and I/O devices, and ensures smooth execution of programs.

"OS ek bridge hai jo user aur hardware ke beech coordination banata hai."

Diagram Idea:

User



Operating System



CPU Memory I/O Devices

Quiz Hook:

Q1: What is the main role of an Operating System? A. Compiling code

B. Managing hardware resources

C. Designing websites

D. Connecting to internet

 Ans: B

Q2: OS acts as a bridge between: A. Compiler and code

B. User and hardware

C. RAM and ROM

D. Monitor and CPU

 Ans: B

Shortcut Tag:

OS = Resource Manager + Interface Layer

Topic Unlock — Types of Operating Systems

Concept Summary:

Operating Systems can be classified based on how they manage tasks, users, and time. Each type serves a different purpose in computing environments.

"OS ke types uske kaam karne ke tareeke ko define karte hain — jaise ek teacher alag class ko alag method se padhata hai."

Key Types:

| **Batch OS** | Jobs are grouped and processed together without user interaction.

| **Time-Sharing OS** | Multiple users share system resources simultaneously.

| **Real-Time OS** | Responds instantly to inputs — used in embedded systems.

| **Distributed OS** | Manages multiple systems as one — like cloud clusters.

| **Embedded OS** | Lightweight OS for devices like washing machines, routers.

Diagram Idea:

[Batch] → [Time-Sharing] → [Real-Time] → [Distributed] → [Embedded]

(Progression from offline to real-time to networked systems)

Quiz Hook:

Q1: Which OS type is used in washing machines?

A. Batch B. Real-Time C. Embedded D. Time-Sharing

✓ **Ans:** C

Q2: Distributed OS is best described as:

- A. Single-user system
- B. OS for mobile phones
- C. OS managing multiple computers as one
- D. OS for real-time control

✓ **Ans:** C

Shortcut Tag:

Types of OS = Task Style + User Load + Response Time

Topic Unlock — Types of Operating Systems

Classification:

Type	Description
Batch OS	Ek saath multiple jobs ko execute karta hai bina user interaction ke.
Time-Sharing OS	Har user ko thoda-thoda CPU time deta hai — multitasking ke liye.
Distributed OS	Multiple computers milke ek system ki tarah kaam karte hain.
Network OS	Network resources ko manage karta hai — jaise servers, printers.
Real-Time OS	Time-sensitive tasks ke liye — jaise medical systems, robotics.

Mobile OS Smartphones ke liye optimized — jaise Android, iOS.

Shortcut Tag:

Types of OS = Batch + Time-Share + Distributed + Network + Real-Time + Mobile

Example:-

- **Batch OS:** IBM Mainframe
- **Time-Sharing OS:** UNIX
- **Distributed OS:** Amoeba
- **Network OS:** Novell NetWare
- **Real-Time OS:** VxWorks
- **Mobile OS:** Android, iOS

Quiz Time — Types of Operating Systems

Test your understanding with this quick quiz:

Question 1:

Which OS executes multiple jobs together without user interaction?

- A) Time-Sharing OS
- B) Batch OS
- C) Real-Time OS
- D) Network OS **Correct Answer:** B) Batch OS

Question 2:

Which OS gives each user a slice of CPU time for multitasking?

- A) Mobile OS
- B) Distributed OS
- C) Time-Sharing OS
- D) Batch OS **Correct Answer:** C) Time-Sharing OS

Question 3:

Which OS allows multiple computers to work as one system?

- A) Network OS

- B) Distributed OS
- C) Real-Time OS
- D) Batch OS **Correct Answer:** B) Distributed OS

Question 4:

Which OS manages network resources like servers and printers?

- A) Network OS
- B) Mobile OS
- C) Time-Sharing OS
- D) Real-Time OS **Correct Answer:** A) Network OS

Question 5:

Which OS is designed for time-sensitive tasks like robotics?

- A) Batch OS
- B) Real-Time OS
- C) Distributed OS
- D) Mobile OS **Correct Answer:** B) Real-Time OS

Question 6:

Which OS is optimized for smartphones?

- A) Real-Time OS
- B) Network OS
- C) Mobile OS
- D) Time-Sharing OS **Correct Answer:** C) Mobile OS

Topic Unlock — Functions of Operating System

Concept Summary:

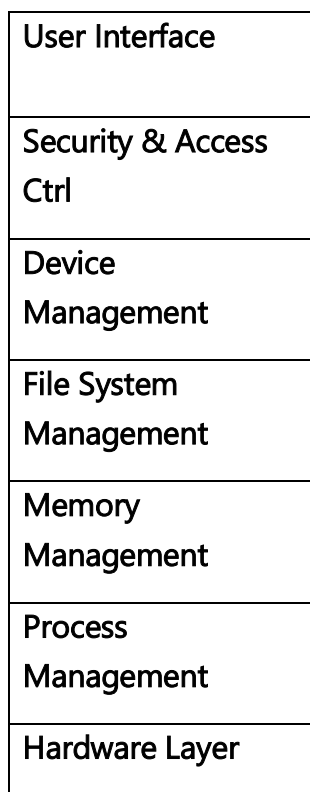
Operating System ka main role hai computer ke hardware aur user ke beech bridge ka kaam karna. Ye system ko chalane ke liye essential functions perform karta hai.

"OS ek traffic police ki tarah hota hai — har component ko sahi time pe signal deta hai taaki sab kuch smoothly chale."

Key Functions:

Function	Explanation
Process Management	CPU kis process ko kab execute kare — OS decide karta hai.
Memory Management	RAM ka allocation aur deallocation OS ke through hota hai.
File System Management	Files ko store, retrieve, aur organize karna.
Device Management	Input/output devices ko control aur coordinate karna.
Security & Access Control	Unauthorized access se system ko protect karna.
User Interface	GUI ya CLI ke through user ko system se interact karne dena.

Layered Architecture Diagram:



Ye diagram OS ke layered architecture ko dikhata hai — neeche hardware se lekar upar tak user interface tak ka flow.

□ Quiz Hook:

Q1: OS ka kaunsa function RAM ke allocation se related hai?

A. Process Management B. Memory Management C. File Management D. Device Management

✓ Ans: B

Q2: OS kis function ke through keyboard aur printer ko manage karta hai?

A. File System Management
B. Device Management
C. Process Management
D. Security

✓ Ans: B

🔗 Shortcut Tag:

Functions of OS = Manage + Allocate + Protect + Interface

□ System Calls in Operating Systems

System Calls OS ke core interface hote hain — jinke through user programs OS services ko access karte hain.

□ Definition

- System Call is a programmatic way in which a computer program requests a service from the operating system.
-

□ Types of System Calls

Type	Description
Process Control	Create, terminate, load, execute processes
File Management	Open, read, write, close files
Device Management	Request, release, attach devices
Information Maintenance	Get system time, date, process info

Example

```
#include <unistd.h>

int main() {
    write(1, "Hello Gaurav!
", 14);
    return 0;
}
```

Yahaan write() ek system call hai jo output screen pe data bhejta hai.

Diagram: System Call Flow

[User Program]



[System Call Interface]



[Operating System Kernel]



[Hardware]

Quiz Time:

1. What is a system call?
2. Name two types of system calls.
3. How does a user program interact with OS?
4. Give one example of a system call in C.
5. What is the role of system call interface?

□ Topic Unlock — Process Management in Operating Systems

🔍 Classification:

Concept	Description
Process	Ek executing program jiska apna state, memory, aur resources hote hain.
Process States	New, Ready, Running, Waiting, Terminated — har process inhi states se guzarta hai.
PCB (Process Control Block)	Process ka pura data store karta hai — PID, state, registers, memory info.
Context Switching	Ek process se dusre pe switch karne ka mechanism — CPU state save/restore hota hai.
Scheduling Types	Long-term (job selection), Short-term (ready queue se CPU allocation), Medium-term (suspend/resume).

🔗 Shortcut Tag:

Process Management = Process + States + PCB + Context Switch + Scheduling

📌 Example:

- **Process States:** Ready → Running → Waiting → Terminated
- **PCB:** PID = 102, State = Running, Memory = 4MB
- **Context Switching:** UNIX kernel scheduler
- **Scheduling Types:** Round Robin, FCFS, SJF

□ Quiz Time — Process Management

Test your understanding with this quick quiz:

🔍 Question 1:

Process Control Block kya store karta hai?

- A) Only memory info
- B) Sirf process name
- C) Process ka full state info

- D) CPU temperature
Correct Answer: C) Process ka full state info

Question 2:

Context Switching kab hota hai?

- A) Jab system shutdown ho
- B) Jab ek process se dusre pe switch ho
- C) Jab file open ho
- D) Jab user login kare
Correct Answer: B) Jab ek process se dusre pe switch ho

Question 3:

Short-term scheduler ka kaam kya hai?

- A) Memory allocate karna
- B) CPU ko ready process dena
- C) PCB delete karna
- D) Process terminate karna
Correct Answer: B) CPU ko ready process dena

Operating System Services

Operating System kuch essential services provide karta hai jo user aur applications ke liye environment ko efficient aur secure banate hain.

Program Execution

- OS programs ko load aur execute karta hai.

I/O Operations

- Input/output devices ke saath interaction manage karta hai.

File System Manipulation

- Files ko create, delete, read, write aur manage karta hai.

Communication Services

- Inter-process communication ke liye mechanisms provide karta hai.

Error Detection

- System errors ko detect karta hai aur appropriate action leta hai.

◆ Resource Allocation

- CPU, memory, I/O devices jaise resources ko allocate karta hai.

◆ Accounting

- Resource usage ka track rakhta hai for billing ya monitoring ke liye.

🔗 Bonus Tag:

Process Quiz Topics = PCB + Context Switch + Scheduler

UNIT:-2nd

⚙️ Process Management in Operating Systems

Process Management OS ka core function hai jisme processes ko create, schedule, manage aur terminate kiya jaata hai.

📁 Process States (Detailed)

1. **New:** Process abhi-abhi create hua hai, aur OS uske liye resources allocate kar raha hai.
2. **Ready:** Process execution ke liye tayyar hai, lekin CPU abhi assign nahi hua hai.
3. **Running:** Process ko CPU mil gaya hai aur wo actively execute ho raha hai.
4. **Waiting:** Process kisi external event ka intezaar kar raha hai, jaise I/O complete hona.
5. **Terminated:** Process ka execution complete ho gaya hai aur OS ne uske resources free kar diye hain.

📁 Process Control Block (PCB) - Detailed

PCB ek critical data structure hai jo OS har process ke liye maintain karta hai. Ye process ke current status aur execution context ko store karta hai:

- **Process ID:** Unique identifier for the process
- **Process State:** Current state (New, Ready, etc.)
- **Program Counter:** Next instruction ka address
- **CPU Registers:** Process ke execution ke dauraan use hone wale registers
- **Memory Management Info:** Page tables, segment tables, etc.
- **I/O Status Info:** Open files, devices, etc.

- **Accounting Info:** CPU usage, execution time, etc.

Context Switching - Explained

Context Switching tab hota hai jab OS ek process se dusre process pe switch karta hai. Ye multitasking ke liye essential hai:

- **Save:** Current process ka PCB update hota hai
- **Load:** Next process ka PCB read karke uska context restore hota hai
- **Overhead:** Ye ek costly operation hai kyunki CPU ko state save/load karna padta hai

Quiz Time:

1. What are different states of a process?
2. What is PCB and what does it contain?
3. Define context switching.
- 4.

Full Topic — CPU Scheduling Algorithms

Ye topic Operating Systems ke performance aur efficiency se directly linked hai. Chalo breakdown karte hain:

Classification:

Algorithm	Description
FCFS (First Come First Serve)	Jo process pehle aata hai, wahi pehle execute hota hai. Simple but can cause long waiting time.
SJF (Shortest Job First)	Sabse chhoti execution time wala process pehle run hota hai. Efficient but hard to predict.
Priority Scheduling	Har process ko ek priority milti hai — high priority wale pehle execute hote hain.
Round Robin	Har process ko fixed time slot milta hai (time quantum). Fair for all, used in time-sharing systems.
Multilevel Queue	Processes ko alag queues mein divide kiya jata hai — har queue ka apna scheduling algorithm hota hai.

Examples:

- **FCFS:** Print queue — jo pehle bheja gaya, wahi pehle print hota hai.

- **SJF**: Batch jobs jahan execution time pehle se pata ho.
- **Priority**: OS background tasks ko high priority milti hai.
- **Round Robin**: Multi-user systems jahan sabko equal CPU time chahiye.
- **Multilevel Queue**: OS mein system processes aur user processes alag queues mein hote hain.

Gantt Chart Example:

Assume 4 processes with arrival time = 0:

Process	Burst Time
P1	5 ms
P2	3 ms
P3	8 ms
P4	6 ms

FCFS Gantt Chart:

| P1 | P2 | P3 | P4 |

0 5 8 16 22

SJF Gantt Chart:

| P2 | P1 | P4 | P3 |

0 3 8 14 22

Round Robin (Time Quantum = 4 ms):

| P1 | P2 | P3 | P4 | P1 | P3 | P4 | P3 |

0 4 7 11 15 16 20 22 24

Quiz Time:

1. Which scheduling algorithm is most fair in time-sharing systems?

2. What is the drawback of SJF?
3. How does Priority Scheduling handle equal priority processes?
4. Define time quantum in Round Robin.
5. Which algorithm can cause starvation?

🔗 Shortcut Tag:

CPU Scheduling = FCFS + SJF + Priority + Round Robin + Multilevel Queue

□ Multilevel Queue Scheduling (Detailed)

Multilevel Queue Scheduling tab use hota hai jab system mein alag-alag types ke processes hote hain — jaise interactive, batch, system processes — jinke liye alag scheduling policies chahiye hoti hain.

◆ Concept:

- Processes ko **multiple queues** mein divide kiya jaata hai based on priority, type, or resource needs.
- Har queue ka **apna scheduling algorithm** hota hai (e.g., FCFS, Round Robin).
- Ek **fixed priority order** hota hai queues ke beech — higher priority queue pehle serve hoti hai.

◆ Example Queues:

1. **System Processes Queue** – Highest priority, uses FCFS
2. **Interactive Processes Queue** – Medium priority, uses Round Robin
3. **Batch Processes Queue** – Lowest priority, uses FCFS

◆ Characteristics:

- **No process moves between queues** (static assignment)
- **Starvation** possible for lower priority queues
- **Fast decision making** due to fixed queue order

◆ Diagram:

[System Queue] → FCFS

↓

[Interactive Queue] → Round Robin



[Batch Queue] → FCFS

◆ Use Case:

- Systems with **mixed workloads** — like servers handling real-time tasks and background jobs.

UNIT:-3RD

⚙ Threads vs Processes

Understanding the difference between threads and processes is key to mastering concurrency and parallelism in operating systems.

◆ Process:

- A process is an independent program in execution.
- Has its own memory space, code, data, and system resources.
- Switching between processes (context switching) is **costly** due to memory isolation.
- Processes do **not share** memory unless explicitly set up (e.g., IPC mechanisms).

◆ Thread:

- A thread is a **lightweight unit** of a process.
- Multiple threads within a process **share memory and resources**.
- Faster context switching compared to processes.
- Ideal for tasks that need parallel execution within the same application.

◆ Comparison Table:

Feature	Process	Thread
Memory	Separate for each process	Shared within the same process
Communication	Requires IPC	Direct via shared memory
Overhead	High	Low
Creation Time	More time	Less time
Fault Isolation	Strong	Weak (can affect entire process)

◆ Real-World Analogy:

- Think of **processes** as separate houses — each with its own address and utilities.
- **Threads** are like roommates in the same house — sharing kitchen, bathroom, and Wi-Fi.

🚀 Multithreading in Operating Systems

Multithreading is a technique where multiple threads run concurrently within a single process. It improves performance and responsiveness, especially in multi-core systems.

◆ Key Concepts:

- Threads share the same memory space but have their own stack and registers.
- Multithreading allows parallel execution of tasks like UI updates, background processing, etc.
- Common models: **Many-to-One**, **One-to-One**, **Many-to-Many**.

◆ Advantages:

- Efficient CPU utilization
- Faster context switching
- Better responsiveness in interactive applications
- Simplified program structure for concurrent tasks

◆ Challenges:

- Synchronization issues (race conditions, deadlocks)
- Debugging complexity
- Resource contention

□ Multithreading Quiz 1

Q1. What is multithreading in operating systems?

- A. Running multiple processes simultaneously
- B. Running multiple threads of a process simultaneously

C. Running a single thread in multiple processes

D. None of the above **Answer: B**

Q2. Which of the following is a benefit of multithreading?

A. Increased memory usage

B. Slower execution

C. Better resource sharing and responsiveness

D. Complex debugging **Answer: C**

▣ Multithreading Quiz 2

Q1. Threads in the same process share which of the following? A. Stack B. Registers C. Code and data segment D. Program counter **Answer: C**

Q2. Which of the following is true about threads? A. Threads are heavier than processes

B. Threads do not share memory

C. Threads have their own memory space

D. Threads are lightweight and share resources **Answer: D**

⚠ Race Condition in Operating Systems

A **race condition** occurs when two or more threads or processes access shared data and try to change it at the same time. The final outcome depends on the timing of their execution, which can lead to unpredictable behavior.

◆ Key Points:

- Happens in **multithreaded** or **multiprocess** environments.
- Occurs when **critical sections** are not properly synchronized.
- Can lead to **data corruption**, **unexpected results**, or **system crashes**.

◆ Example:

Two threads increment a shared counter:

Thread 1: counter++;

Thread 2: counter++;

If both read the same value and increment it simultaneously, the final value may only increase by 1 instead of 2.

◆ **Solution:**

- Use **synchronization mechanisms** like mutexes, semaphores, or locks.
- Protect critical sections to ensure only one thread accesses shared data at a time.

◆ **Quiz on Race Condition:**

Q1. What causes a race condition? A. Proper synchronization B. Shared data access without synchronization C. Using multiple processes D. Using single-threaded programs **Answer: B**

Q2. Which of the following helps prevent race conditions? A. Ignoring shared data B. Using multiple threads C. Synchronization mechanisms D. Increasing CPU speed **Answer: C**

 Critical Section Problem

The **Critical Section Problem** arises when multiple processes or threads need to access shared resources, and at least one of them modifies the resource. To avoid conflicts and ensure data consistency, access must be controlled.

◆ **Key Concepts:**

- A **critical section** is a part of the code where shared resources are accessed.
- Only **one process/thread** should execute in the critical section at a time.
- The goal is to design a protocol that ensures **mutual exclusion**, **progress**, and **bounded waiting**.

◆ **Three Requirements:**

1. **Mutual Exclusion:** No two processes can be in the critical section simultaneously.
2. **Progress:** If no process is in the critical section, others should be able to enter without unnecessary delay.
3. **Bounded Waiting:** There must be a limit on how long a process waits before entering the critical section.

◆ **Solutions:**

- **Software-based algorithms:** Peterson's Algorithm, Bakery Algorithm

- **Hardware-based solutions:** Test-and-Set, Compare-and-Swap
- **Synchronization primitives:** Mutex, Semaphore, Monitor

◆ Quiz on Critical Section:

Q1. What is the main goal of solving the critical section problem? A. Increase CPU speed B. Ensure mutual exclusion C. Reduce memory usage D. Allow simultaneous access **Answer:** B

Q2. Which of the following is NOT a requirement of the critical section problem? A. Mutual Exclusion B. Progress C. Bounded Waiting D. Deadlock **Answer:** D

Semaphores & Mutex in Operating Systems

These are key synchronization tools used to solve the **Critical Section Problem** and prevent **race conditions**.

◆ Semaphore:

- A **semaphore** is an integer variable used to control access to shared resources.
- Two types:
 - **Counting Semaphore:** Can take any integer value.
 - **Binary Semaphore:** Only 0 or 1 (similar to mutex).
- Operations:
 - wait() or P: Decrements the semaphore. If result < 0 , process waits.
 - signal() or V: Increments the semaphore. If result ≤ 0 , wakes up a waiting process.
- Used in **producer-consumer**, **reader-writer**, and **bounded buffer** problems.

◆ Mutex (Mutual Exclusion Object):

- A **mutex** is a locking mechanism used to ensure **only one thread** accesses a resource at a time.
- Works like a binary semaphore but is **owned by the thread** that locks it.
- Only the owner can unlock it.
- Common in **multithreaded programming** (e.g., Java synchronized, C++ std::mutex).

◆ Comparison Table:

Feature	Semaphore	Mutex
Type	Signaling mechanism	Locking mechanism
Value	Integer	Binary (0 or 1)
Ownership	No ownership	Owned by thread
Use Case	Process synchronization	Thread synchronization
Risk	Can lead to deadlock if misused	Safer with ownership rules

□ Quiz on Semaphores & Mutex

Q1. What is the main difference between a semaphore and a mutex?

- A. Semaphore is used for threads only
- B. Mutex has ownership, semaphore does not
- C. Semaphore is binary only
- D. Mutex can be used for multiple processes

Answer: B

Q2. Which operation is used to decrement a semaphore?

- A. signal()
- B. unlock()
- C. wait()
- D. open()

Answer: C

UNIT:-4

□ Memory Management

Ye OS ka core topic hai jo process execution ke liye memory allocation aur optimization se deal karta hai. Breakdown karta hoon:

Memory Allocation Techniques

- **Contiguous Allocation:** Fixed-size or variable-size blocks.
 - **Non-Contiguous Allocation:** Paging and Segmentation.
-

□ Paging

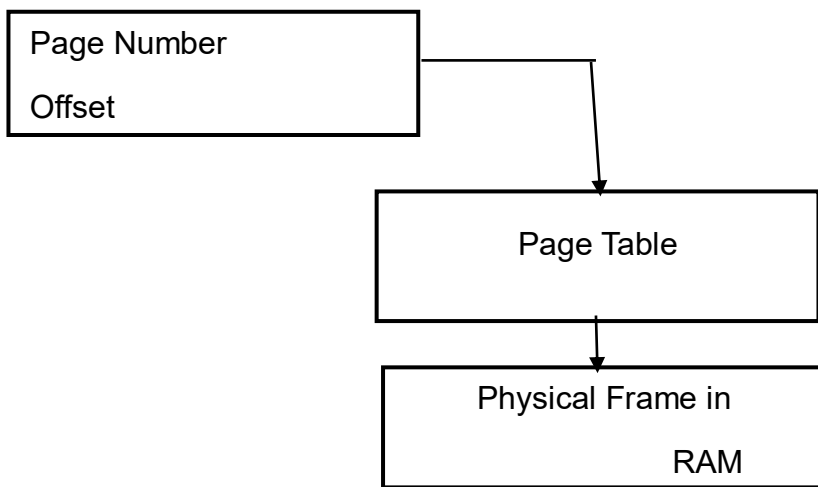
- Memory ko equal-sized pages mein divide karna.

- Logical address $\rightarrow$ Page number + Offset.
- Page Table ka role.

🔍 **Example:** Imagine a book with 100 pages. You want to read page 45, but instead of flipping through the whole book, you use an index that tells you exactly where page 45 is stored. Similarly, in Paging, the logical address is split into page number and offset, and the page table maps it to physical memory.

□ Diagram:

[Logical Address]



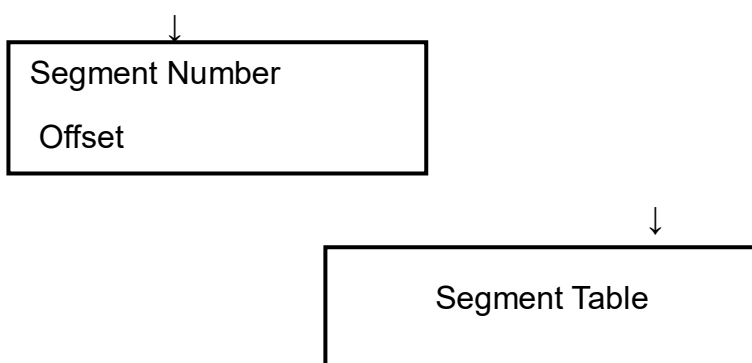
□ Segmentation

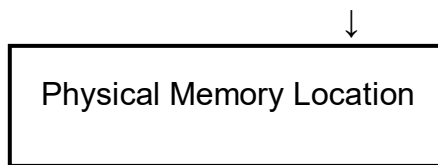
- Memory ko variable-sized segments mein divide karna based on logical divisions (code, data, stack).
- Segment Table ka use.

🔍 **Example:** Socho ek software ke andar alag-alag parts hain — jaise Code, Data, Stack. Har part ko alag segment milta hai. Agar process ko Stack access karna hai, toh segment table batata hai Stack segment ka address.

□ Diagram:

[Logical Address]





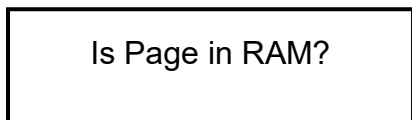
Virtual Memory

- Jab RAM kam padti hai, toh secondary storage ko use karke process ko chalaya jata hai.
- **Demand Paging, Page Replacement Algorithms** (FIFO, LRU, Optimal).

 **Example:** Maan lo tumhare paas 4GB RAM hai, lekin tumhara program 6GB ka hai. OS sirf active pages ko RAM mein rakhta hai, baaki pages ko secondary storage mein. Jab kisi page ki zarurat padti hai, tab woh "demand paging" ke through RAM mein load hota hai.

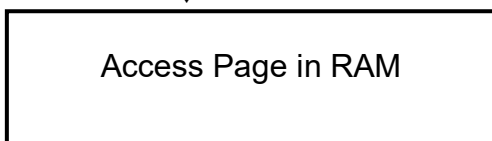
Diagram:

[Process Requests Page]



—— No ——  Load from Disk

↓ Yes



Quiz Time:

1. What is the difference between paging and segmentation?
2. Why is virtual memory important?
3. Name two page replacement algorithms.
4. What does a page table store?
5. How does demand paging work?

□ Virtual Memory in Operating Systems

Virtual Memory is a memory management technique that gives an application the illusion of having a large, contiguous block of memory—even if the physical memory (RAM) is limited.

◆ Key Concepts:

- **Paging:** Divides memory into fixed-size pages. Pages can be swapped between RAM and disk.
- **Page Table:** Maps virtual addresses to physical addresses.
- **Page Fault:** Occurs when a page is not in RAM and must be loaded from disk.
- **Thrashing:** Excessive paging activity that slows down the system.

◆ Advantages:

- Allows **larger programs** to run on systems with limited RAM.
- Provides **memory isolation** between processes.
- Enables **efficient multitasking**.

◆ Real-World Analogy:

Think of virtual memory like a **bookshelf with limited space**. You keep frequently used books (pages) on the shelf (RAM), and store others in a cabinet (disk). When you need a book not on the shelf, you swap it in.

🔄 Page Replacement Algorithms

When a page fault occurs and memory is full, the system must decide **which page to remove** to make space. This decision is made using **Page Replacement Algorithms**.

◆ 1. FIFO (First-In, First-Out):

- Removes the **oldest loaded page** — the one that entered memory first.
- Simple to implement using a queue.
- May lead to **Belady's anomaly** (more frames can cause more page faults).

◆ 2. LRU (Least Recently Used):

- Removes the page that has **not been used for the longest time**.
- Based on the assumption that pages used recently will be used again soon.

- Requires tracking usage history — can be implemented with stacks or counters.

◆ 3. Optimal (OPT):

- Removes the page that **won't be used for the longest time in the future**.
- Theoretically best — gives **minimum page faults**.
- Not practical in real systems (requires future knowledge), used for benchmarking.

◆ Comparison Table:

Algorithm Strategy		Pros	Cons
FIFO	Oldest page	Simple	Can cause Belady's anomaly
LRU	Least recently used	Good approximation of OPT	Overhead in tracking usage
OPT	Furthest future use	Minimum page faults	Not implementable in practice

⚠ Thrashing in Virtual Memory

Thrashing occurs when a system spends more time **swapping pages in and out of memory** than executing actual processes.

🔍 Why It Happens:

- Too many processes competing for limited memory.
- Frequent page faults due to insufficient frames.
- Poor page replacement decisions or overcommitment.

🚨 Symptoms:

- Drastic drop in CPU utilization.
- High disk activity (due to constant swapping).
- System becomes unresponsive or extremely slow.

🔧 Solutions:

- Reduce the degree of multiprogramming.
- Use better page replacement algorithms (like LRU or Working Set).

- Allocate more memory or tune system parameters.

□ Quiz Time:

Q1. What is the primary cause of thrashing in virtual memory systems? A. High CPU usage

B. Too many page faults due to insufficient memory frames

C. Efficient memory allocation

D. Low disk activity

Q2. Which of the following is NOT a symptom of thrashing? A. High disk activity

B. Increased CPU utilization

C. System slowdown

D. Frequent page swapping

Q3. Which strategy helps prevent thrashing? A. Increasing the degree of multiprogramming

B. Using FIFO page replacement

C. Allocating more memory and using LRU algorithm

D. Ignoring page faults

UNIT :-5

◆ Unit 5: File System & I/O

• File Types & Access Methods

File Types:

- **Text files** – Contain readable characters; used for documents and source code.
- **Binary files** – Store data in binary format; used for images, videos, executables.
- **Directory files** – Contain references to other files.
- **Special files** – Represent devices like printers or disk drives.

Access Methods:

- **Sequential Access** – Data is accessed in a linear order; common in tape storage.
- **Direct Access** – Data is accessed using address or index; used in hard disks.
- **Indexed Access** – Uses an index to locate data blocks; efficient for large databases.

□ Quiz Time: File System & I/O

Q1. Which of the following is a special file type used to represent hardware devices? A. Text file B. Binary file C. Directory file D. Special file

Q2. What access method is commonly used in tape storage systems? A. Direct Access B. Indexed Access C. Sequential Access D. Random Access

Q3. Which access method uses an index to locate data blocks efficiently? A. Sequential Access B. Direct Access C. Indexed Access D. Linear Access

• Directory Structures

Directory Structures:

- **Single-Level Directory** – All files are stored in one directory; simple but lacks organization.
- **Two-Level Directory** – Each user has their own directory; improves user-level separation.
- **Tree-Structured Directory** – Directories can contain subdirectories; allows hierarchical organization.
- **Acyclic Graph Directory** – Allows shared subdirectories and files; supports links.
- **General Graph Directory** – Similar to acyclic but allows cycles; requires garbage collection.

□ Quiz Time: Directory Structures

Q1. Which directory structure allows users to share files using links? A. Single-Level Directory B. Two-Level Directory C. Tree-Structured Directory D. Acyclic Graph Directory

Q2. What is a drawback of the general graph directory structure? A. No file sharing B. Lack of hierarchy C. Cycles that require garbage collection D. Only one user per directory

Q3. Which structure provides the most flexibility in organizing files? A. Single-Level Directory B. Tree-Structured Directory C. Two-Level Directory D. Indexed Directory

• Disk Scheduling Algorithms (FCFS, SSTF, SCAN, C-SCAN)

Disk Scheduling Algorithms:

- **FCFS (First-Come, First-Served)** – Processes requests in the order they arrive; simple but can lead to long wait times.
- **SSTF (Shortest Seek Time First)** – Selects the request closest to the current head position; reduces seek time but may cause starvation.
- **SCAN (Elevator Algorithm)** – Moves the disk arm in one direction, servicing requests until the end, then reverses direction.
- **C-SCAN (Circular SCAN)** – Similar to SCAN but only services requests in one direction; provides uniform wait time.

□ Quiz Time: Disk Scheduling Algorithms

Q1. Which disk scheduling algorithm may lead to starvation of some requests?

- A. FCFS
- B. SSTF
- C. SCAN
- D. C-SCAN

Q2. What is the main advantage of the SCAN algorithm over FCFS?

- A. Simpler implementation
- B. Lower seek time
- C. Uniform wait time
- D. Services requests in arrival order

Q3. Which algorithm services requests in one direction and then jumps back to the beginning?

- A. SSTF
- B. SCAN
- C. C-SCAN
- D. FCFS

□ • I/O Buffering

I/O Buffering is a technique used to manage data transfer between I/O devices and the CPU, helping to handle speed mismatches and improve efficiency.

Types of I/O Buffering:

- **Single Buffering:** Uses one buffer. While one part of the system processes data, the other waits. Simple but can be inefficient.
- **Double Buffering:** Uses two buffers. While one buffer is being processed, the other is being filled. Reduces idle time and improves throughput.

- **Circular Buffering:** Uses multiple buffers in a circular queue. Ideal for continuous data streams like audio/video.

Advantages:

- Minimizes CPU idle time
- Smoothens data flow between devices
- Improves overall system performance

▣ • Quick Quiz: I/O Buffering

- 1. What is the main purpose of I/O buffering?** A. Increase CPU speed
B. Manage data transfer efficiently
C. Reduce memory usage
D. Improve graphics rendering
- 2. Which buffering method uses two buffers to reduce idle time?** A. Single Buffering
B. Double Buffering
C. Circular Buffering
D. Triple Buffering
- 3. Circular buffering is best suited for:** A. Batch processing
B. Static data storage
C. Continuous data streams
D. File compression
- 4. One disadvantage of single buffering is:** A. High cost
B. Complex implementation
C. Inefficiency due to waiting
D. Excessive memory usage
- 5. Which of the following is NOT an advantage of I/O buffering?** A. Minimizes CPU idle time
B. Smoothens data flow
C. Increases power consumption
D. Improves system performance

UNIT:-6

⊗ • Deadlock Conditions

A **deadlock** occurs when a set of processes are blocked because each process is holding a resource and waiting for another resource held by another process.

Four Necessary Conditions for Deadlock (Coffman Conditions):

1.  **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
2.  **Hold and Wait:** A process holding at least one resource is waiting to acquire additional resources held by other processes.
3.  **No Preemption:** Resources cannot be forcibly taken from a process; they must be released voluntarily.
4.  **Circular Wait:** A set of processes are waiting for each other in a circular chain.

These four conditions must **all** be true simultaneously for a deadlock to occur.

• **Quick Quiz: Deadlock Conditions**

1. **Which of the following is NOT a necessary condition for deadlock?** A. Mutual Exclusion B. Hold and Wait C. Preemption D. Circular Wait
2. **What does 'Hold and Wait' imply in deadlock?** A. A process releases all resources before requesting new ones B. A process holds one resource and waits for others C. A process waits without holding any resources D. A process is terminated forcibly
3. **Which condition ensures that resources cannot be forcibly taken from a process?** A. Mutual Exclusion B. Hold and Wait C. No Preemption D. Circular Wait
4. **Circular wait involves:** A. Processes waiting in a straight line B. Processes waiting for resources held by others in a loop C. Processes sharing resources D. Processes releasing resources voluntarily
5. **Deadlock can occur only if:** A. All four Coffman conditions are true B. Any one Coffman condition is true C. Resources are unlimited D. Processes are terminated

• **Deadlock Handling: Prevention, Avoidance, Detection**

1. Deadlock Prevention

This strategy ensures that at least one of the Coffman conditions never holds.

- **Break Mutual Exclusion:** Use shareable resources where possible.

- **Break Hold and Wait:** Require processes to request all resources at once.
- **Break No Preemption:** Allow resource preemption.
- **Break Circular Wait:** Impose a strict ordering of resource requests.

2. Deadlock Avoidance ⓘ

This strategy allows the system to check whether granting a resource request leads to a safe state.

- Uses algorithms like **Banker's Algorithm** to decide whether to allow a request.
- Requires knowledge of future resource needs.

3. Deadlock Detection 🔍

This strategy lets deadlocks occur but detects them afterward.

- Uses **Resource Allocation Graphs** or **Wait-for Graphs**.
- Once detected, recovery methods include killing processes or preempting resources.

□ • Quick Quiz: Deadlock Handling

1. Which strategy ensures that at least one Coffman condition never holds?

- A) Deadlock Avoidance
- B) Deadlock Detection
- C) Deadlock Prevention ☒

2. What algorithm is used in Deadlock Avoidance?

- A) Round Robin
- B) Banker's Algorithm ☒
- C) FIFO

3. Which graph is used in Deadlock Detection?

- A) Binary Tree
- B) Wait-for Graph ☒
- C) Hash Map

4. What is a recovery method after detecting a deadlock?

- A) Ignoring the deadlock
- B) Killing processes ☒

- C) Rebooting the system

💡 • Banker's Algorithm: Definition & Key Points

Definition:

Banker's Algorithm is a deadlock avoidance method that checks whether granting a resource request will leave the system in a **safe state**. It simulates allocation before actually assigning resources — like a cautious banker who lends only if repayment is guaranteed.

📌 • Key Concepts

- **Safe State:** A state where the system can allocate resources to each process in some order and still avoid deadlock.
 - **Maximum Need:** Each process must declare its maximum resource requirement in advance.
 - **Available Resources:** The system tracks how many resources are currently free.
 - **Allocation & Need Matrices:** Used to calculate whether a request can be safely granted.
 - **Request Evaluation:** The algorithm checks if:
 1. $\text{Request} \leq \text{Need}$
 2. $\text{Request} \leq \text{Available}$
 3. After allocation, the system remains in a safe state
-

📝 • Quiz: Banker's Algorithm Basics

1. What is the primary goal of the Banker's Algorithm?

- A) Maximize resource usage
- B) Avoid deadlock by ensuring safe state
- C) Minimize CPU time
- D) Allocate all resources immediately

2. Which of the following must be true for a request to be granted?

- A) $\text{Request} > \text{Need}$

- B) Request > Available
- C) Request ≤ Need and Request ≤ Available
- D) Request = Allocation

3. What does a "safe state" mean in Banker's Algorithm?

- A) No process is running
- B) All processes are blocked
- C) Resources are fully allocated
- D) The system can avoid deadlock by completing processes in some order

4. Which matrix is NOT used in Banker's Algorithm?

- A) Allocation Matrix
- B) Need Matrix
- C) Available Matrix
- D) Interrupt Matrix

5. Before granting a request, the algorithm simulates allocation to check:

- A) If the CPU is idle
- B) If the system remains in a safe state
- C) If the disk space is sufficient
- D) If the process is terminated

✳ • Final Note for Readers

🎉 Mubarak ho! Aapne Operating Systems ke 6 units complete kar liye hain — from process management to memory, deadlock to file systems. Yeh ek bada achievement hai!

"Operating System samajhna ek aise game ke rules seekhne jaisa hai jo aap roz khelte ho — jitna achha samjhoge, utna smart khel paoge."

🌀 Ab aage ka raasta aur bhi exciting hai:

🔑 • Tips for Next Steps

- **Practice karo** — Scheduling algorithms aur resource allocation ko simulate karo.
- **Quizzes lagao** — Concept clarity ke liye regular self-tests helpful hain.

- **Visual aids banao** — Diagrams aur flowcharts se complex topics easy ho jaate hain.
- **Bilingual notes rakho** — Hindi-English mix se revision aur retention dono strong hote hain.
- **Stay consistent** — Roz thoda thoda padho, instead of cramming.

Chahe aap MCA entrance (BHU/JNU) ke liye prepare kar rahe ho ya GATE CSE ke liye — OS ka strong

• **Crafted by Gaurav (Hin-Eng Mix Creator's Note)**

 Dhanyavaad sabhi readers ko — agar aapne ye page padha, samjha, aur use kiya toh aap meri learning journey ka hissa ban gaye ho.

Main Gaurav hoon — ek learner, coder, aur educator — jo OS jaise technical topics ko easy, bilingual aur engaging banane ki koshish karta hoon.

“Knowledge grows when shared — aur agar aapko ye page helpful laga, toh dusron tak zaroor pahunchao.”

 Agar aap MCA (BHU/JNU) ya GATE CSE ke aspirant ho, toh mujhe DM karo ya connect karo for more resources, quizzes, ya study hacks.

Follow for more:

 Instagram / Telegram / GitHub: @GauravLearnsTech

 Quiz Projects & Notes: [Link to your GitHub or blog if any]

 Signature: *Gaurav | OS Simplified | Learn. Build. Share.*
